

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Arquitectura Cliente Servidor

Ortega de la Paz Rafael | 420054085

17 de Septiembre del 2026

Tarea 3: fork(), exec() y manejo de errores en llamadas al sistema

Introducción

En el presente trabajo se documenta la ejecución práctica de los ejercicios propuestos en la Tarea 3 de la asignatura, relacionados con el paso de argumentos a un proceso (`argv / argc`), el reemplazo de la imagen de un proceso con la familia de funciones `exec` , y la creación de procesos hijos con `fork()` combinada con `execvp()` . En los tres programas se corrigió el manejo de errores de la llamada a `exec` siguiendo el patrón visto en clase: envolver la llamada en un bloque `if` , reportar la causa exacta del fallo con `perror()` y terminar el proceso con `exit(EXIT_FAILURE)` . El código de los tres programas se tomó del sitio del profesor (`profesores.fi-b.unam.mx/carlos/acs`), se comentó línea por línea y se compiló y ejecutó para obtener evidencia real de su comportamiento.

Actividad 1 — myecho y execve

El programa `programa20_myecho.c` es un ejemplo mínimo para observar cómo el shell le entrega los argumentos a un proceso: recorre el arreglo `argv[]` desde el índice 0 (el propio nombre del ejecutable) hasta `argc-1` e imprime cada uno. A continuación se muestra el código completo, comentado en detalle:

```
#include <stdio.h>    // printf()
#include <stdlib.h>    // exit(), EXIT_SUCCESS

int main(int argc, char *argv[])
{
    // argc (argument count) = número total de argumentos, INCLUYENDO
    // el nombre del propio ejecutable (argv[0]).
    // argv (argument vector) = arreglo de cadenas con cada argumento;
    // argv[argc] siempre vale NULL.
    //
    // Ejemplo: si se ejecuta ./programa20_myecho hola mundo
    // el shell arma:  argc = 3
```

```

//          argv[0] = "./programa20_myecho"
//          argv[1] = "hola"
//          argv[2] = "mundo"
//          argv[3] = NULL  (centinela, no cuenta en argc)

int j;

// Recorremos el arreglo de argumentos desde 0 (el nombre del
// programa) hasta argc-1 (el último argumento real).
for ( j = 0; j < argc; j++ )
{
    // %d imprime el índice j; %s imprime la cadena argv[j] tal
    // cual llegó del shell.
    printf("argv[%d]: %s\n", j, argv[j]);
}

// exit(EXIT_SUCCESS) termina el proceso indicando éxito (código 0)
// y hace flush de los buffers de stdio antes de terminar.
exit( EXIT_SUCCESS );
}

```

Código 1 — programa20_myecho.c comentado

El programa `programa21_execve.c` recibe como único argumento la ruta de otro ejecutable y lo arranca con `execve()`, reemplazando por completo la imagen del proceso actual. En el código original, después de la llamada a `execve()` había un `printf()` que decía "Esta sentencia no se ejecuta", pero el propio comentario del profesor señalaba que eso era falso: tal como estaba el código, si `execve()` fallaba, el flujo seguía de largo y ese `printf()` sí llegaba a imprimirse. La corrección consistió en envolver la llamada en un bloque `if` (mismo patrón usado para `execl()` en clase) y eliminar ese `printf()` engañoso, de modo que, ahora sí, esa sentencia genuinamente nunca se ejecuta:

```

#include <stdio.h>    // fprintf(), perror()
#include <stdlib.h>    // exit(), EXIT_SUCCESS, EXIT_FAILURE
#include <unistd.h>    // execve()

int main(int argc, char *argv[])
{
    // Argumentos que se le pasarán al NUEVO programa que reemplazará
    // a este proceso. newargv[0] se rellena más abajo con el nombre

```

```

// del ejecutable a lanzar; el arreglo debe terminar en NULL.
char *newargv[] = { NULL, "hola", "mundo", "buen", "dia", NULL };

// Este programa exige exactamente 2 argumentos:
// argv[0] = nombre de este ejecutable
// argv[1] = ruta del programa que se quiere ejecutar
if ( argc != 2 )
{
    fprintf( stderr, "La sintaxis es: %s <archivo-a-ejecutar>\n",
argv[0] );
    exit( EXIT_FAILURE );
}

// ESTA SENTENCIA ES MUY IMPORTANTE: por convención POSIX, argv[0]
// del proceso reemplazado debe ser el nombre con el que se le
// invoca, aunque execve() ya sabe, por la ruta, qué binario cargar.
newargv[0] = argv[1];

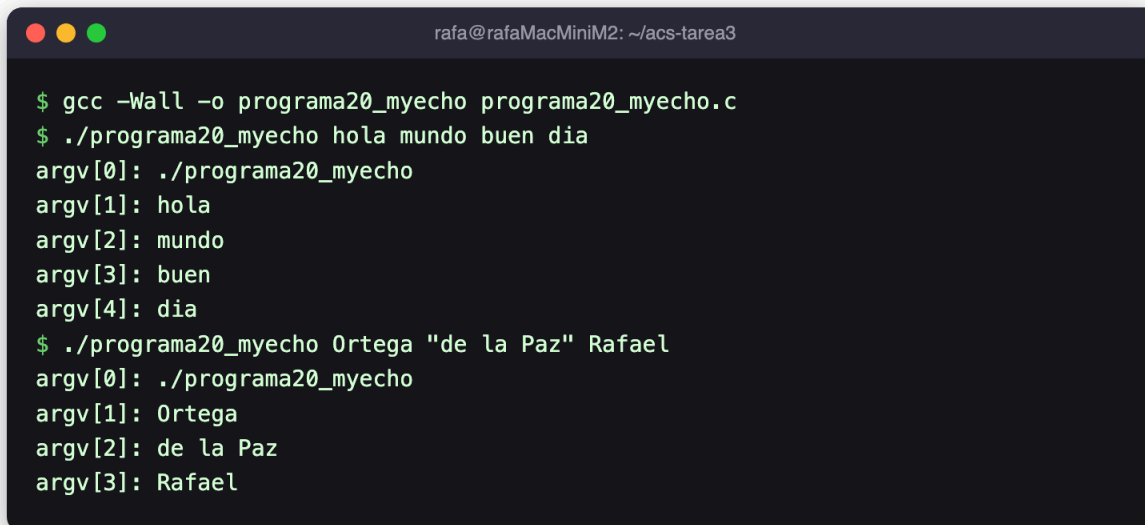
// execve(ruta, argv_nuevo, entorno):
// - ruta = argv[1] (execve() NO busca en $PATH, a diferencia de
execvp())
// - argv_nuevo = newargv
// - entorno = NULL -> el nuevo proceso no hereda variables de
entorno
//
// Si execve() tiene ÉXITO, la imagen de ESTE proceso es reemplazada
// por completo por la del nuevo programa: el código de abajo JAMÁS
// se alcanza en ese caso. execve() solo "regresa" (con -1) si
falló.
if ( execve( argv[1], newargv, NULL ) == -1 )
{
    // Solo llegamos aquí si execve() falló: reportamos la causa
    // exacta (errno) con perror() y terminamos con error.
    perror("execve");
    exit( EXIT_FAILURE );
}

return 0;
}

```

Código 2 — programa21_execve.c comentado, con la corrección del bloque
if/exit(EXIT_FAILURE)

Con el código anterior ya corregido, se compilaron ambos programas y se ejecutó `programa20_myecho` directamente desde la terminal, primero con argumentos genéricos y después con el nombre propio en el formato solicitado (apellido paterno, apellido materno, nombre):

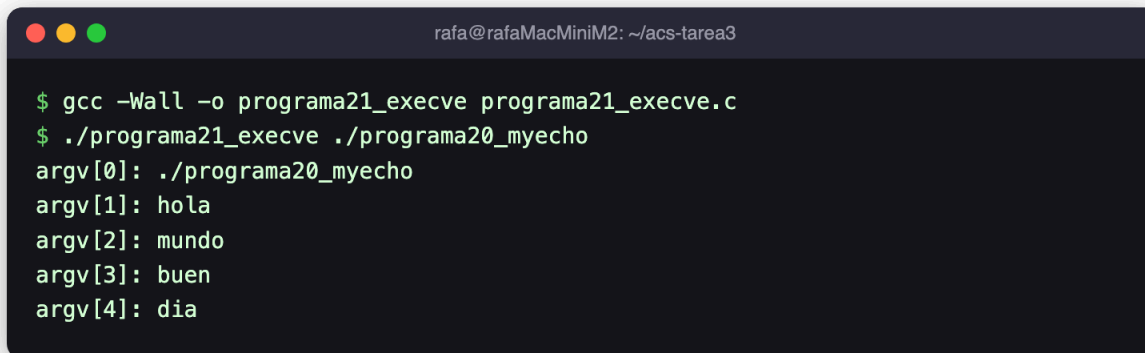


```
rafa@rafaMacMiniM2: ~/acs-tarea3

$ gcc -Wall -o programa20_myecho programa20_myecho.c
$ ./programa20_myecho hola mundo buen dia
argv[0]: ./programa20_myecho
argv[1]: hola
argv[2]: mundo
argv[3]: buen
argv[4]: dia
$ ./programa20_myecho Ortega "de la Paz" Rafael
argv[0]: ./programa20_myecho
argv[1]: Ortega
argv[2]: de la Paz
argv[3]: Rafael
```

Imagen 1 — Ejecución de programa20_myecho con argumentos genéricos y con el nombre en formato apellidoPaterno apellidoMaterno nombre(s)

Finalmente se ejecutó `programa20_myecho` invocándolo desde `programa21_execve`, pasándole su ruta como argumento:

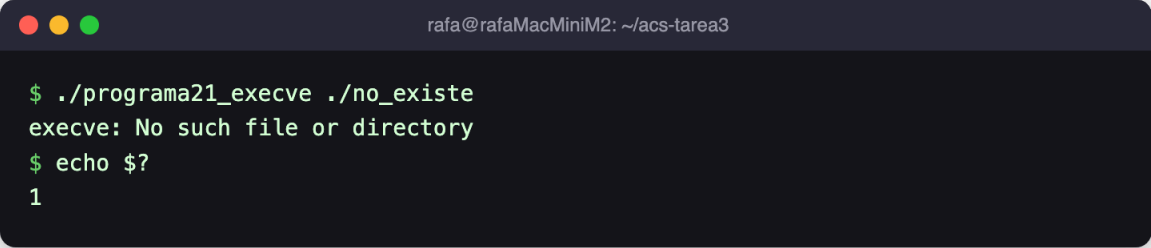


```
rafa@rafaMacMiniM2: ~/acs-tarea3

$ gcc -Wall -o programa21_execve programa21_execve.c
$ ./programa21_execve ./programa20_myecho
argv[0]: ./programa20_myecho
argv[1]: hola
argv[2]: mundo
argv[3]: buen
argv[4]: dia
```

Imagen 2 — Ejecución de programa21_execve ./programa20_myecho: execve() reemplaza la imagen del proceso por la de myecho, que recibe el argv hardcodeado dentro de programa21 (hola, mundo, buen, dia) y como argv[0] el valor de newargv[0] = "./programa20_myecho"

Como verificación adicional del manejo de errores, se probó también la ruta de falla intencional pasando un ejecutable inexistente, confirmando que el bloque `if` corregido funciona como se esperaba:

A terminal window with a dark background and light green text. The window title is 'rafa@rafaMacMiniM2: ~/acs-tarea3'. The commands and output are: '\$./programa21_execve ./no_existe', 'execve: No such file or directory', '\$ echo \$?', and '1'.

```
rafa@rafaMacMiniM2: ~/acs-tarea3
$ ./programa21_execve ./no_existe
execve: No such file or directory
$ echo $?
1
```

Imagen 3 — Verificación del manejo de errores: al fallar `execve()` se reporta la causa exacta con `perror()` y el proceso termina con código de salida 1 (`EXIT_FAILURE`)

Actividad 2 — `fork()` + `execvp()`

`programa23_fork-exec.c` combina `fork()` y `execvp()` dentro de una función auxiliar `spawn()`: crea un proceso hijo y, dentro de ese hijo, reemplaza su imagen por la de otro programa, sin que el padre llame a `wait()` para esperarlo. En el código original, si `execvp()` fallaba, el hijo solo imprimía un mensaje con `printf()` y hacía `return 1;`, sin usar el patrón de manejo de errores visto en clase. La corrección envuelve la llamada a `execvp()` en un bloque `if`, reporta el error con `perror()` y termina el proceso hijo con `exit(EXIT_FAILURE)`:

```
#include <stdio.h>           // printf()
#include <stdlib.h>          // exit(), EXIT_SUCCESS, EXIT_FAILURE
#include <sys/types.h>       // pid_t
#include <unistd.h>          // fork(), execvp()

// spawn: crea un proceso hijo que ejecuta "program" con los
// argumentos de "arg_list". Regresa el PID del hijo al padre.
int spawn(char *program, char **arg_list)
{
    pid_t child_pid;

    // fork() duplica el proceso actual. A partir de aquí hay DOS
    // procesos corriendo el mismo código, cada uno con su copia de
    // child_pid:
    // - en el PADRE, fork() regresa el PID del hijo (>0).
    // - en el HIJO, fork() regresa 0.
```

```

child_pid = fork();

if ( child_pid != 0 )
{
    // Rama del PADRE: regresa el PID del hijo sin esperarlo
    // (no hay wait()).
    return child_pid;
}
else
{
    // Rama del HIJO (child_pid == 0).
    // RECUERDEN: si execvp() se ejecuta exitosamente, la imagen de
    // ESTE proceso hijo es reemplazada por la del programa indicado;
    // el código que sigue a execvp() nunca se alcanza en ese caso.
    //
    // execvp(program, arg_list): a diferencia de execve(), SÍ busca
    // "program" en las rutas de la variable de entorno $PATH.
    // Solo regresa (con -1) si falla.
    if ( execvp(program, arg_list) == -1 )
    {
        // Reportamos la causa exacta del fallo y terminamos el
        // proceso HIJO con código de error.
        perror("execvp");
        exit( EXIT_FAILURE );
    }

    return 1; // inalcanzable: éxito reemplaza el proceso, error hace
    exit() arriba
}
}

int main(void)
{
    // Argumentos para "ls -l /", equivalente a teclear "ls -l /".
    // (Nótese: es la letra "ele" minúscula, no el número "uno".)
    char *arg_list[] = { "ls", "-l", "/", NULL };

    // Lanza "ls -l /" en un hijo. Como no se llama a wait(), el padre
    // sigue su propio flujo sin esperar a que el hijo termine.
    spawn("ls", arg_list);

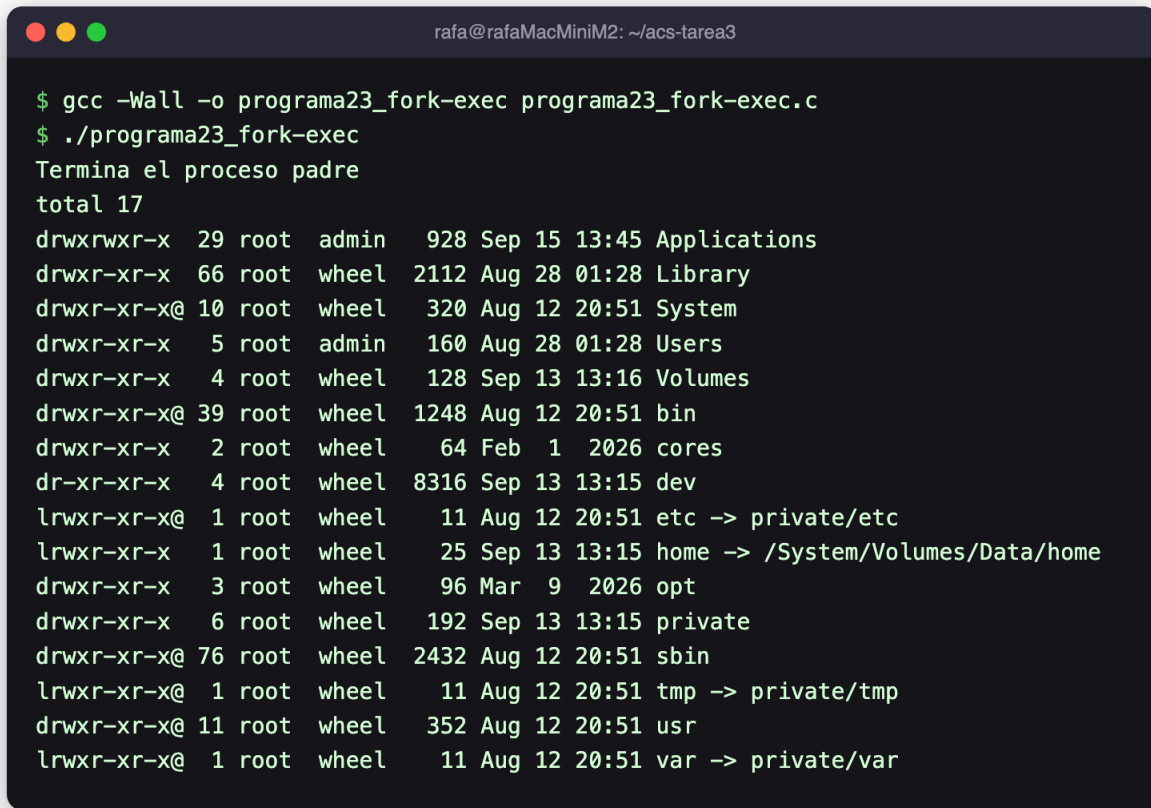
    printf("Termina el proceso padre\n");
}

```

```
return 0;
}
```

Código 3 — programa23_fork-exec.c comentado, con la corrección del bloque if/exit(EXIT_FAILURE) en execvp()

Se compiló y ejecutó el programa, obteniendo la siguiente evidencia real:



```
rafa@rafaMacMiniM2: ~/acs-tarea3
$ gcc -Wall -o programa23_fork-exec programa23_fork-exec.c
$ ./programa23_fork-exec
Termina el proceso padre
total 17
drwxrwxr-x 29 root  admin   928 Sep 15 13:45 Applications
drwxr-xr-x 66 root  wheel  2112 Aug 28 01:28 Library
drwxr-xr-x@ 10 root  wheel   320 Aug 12 20:51 System
drwxr-xr-x  5 root  admin   160 Aug 28 01:28 Users
drwxr-xr-x  4 root  wheel   128 Sep 13 13:16 Volumes
drwxr-xr-x@ 39 root  wheel  1248 Aug 12 20:51 bin
drwxr-xr-x  2 root  wheel    64 Feb  1  2026 cores
dr-xr-xr-x  4 root  wheel  8316 Sep 13 13:15 dev
lrwxr-xr-x@  1 root  wheel    11 Aug 12 20:51 etc -> private/etc
lrwxr-xr-x  1 root  wheel    25 Sep 13 13:15 home -> /System/Volumes/Data/home
drwxr-xr-x  3 root  wheel    96 Mar  9  2026 opt
drwxr-xr-x  6 root  wheel   192 Sep 13 13:15 private
drwxr-xr-x@ 76 root  wheel  2432 Aug 12 20:51 sbin
lrwxr-xr-x@  1 root  wheel    11 Aug 12 20:51 tmp -> private/tmp
drwxr-xr-x@ 11 root  wheel   352 Aug 12 20:51 usr
lrwxr-xr-x@  1 root  wheel    11 Aug 12 20:51 var -> private/var
```

Imagen 4 — Ejecución de programa23_fork-exec: el listado de "ls -l /" corresponde al sistema de archivos del entorno de trabajo usado para compilar y probar el ejercicio

El orden de la salida es evidencia directa del comportamiento asíncrono de `fork()`: como `spawn()` no llama a `wait()`, el proceso padre no espera a que el hijo termine antes de continuar. Por eso el mensaje "Termina el proceso padre" (impreso por el padre inmediatamente después de que `spawn()` regresa) aparece *antes* que el listado de `ls -l /` (impreso por el hijo, que tardó una fracción de segundo más en completar su `execvp()`): ambos procesos corrieron en paralelo, sin sincronización entre ellos.

Conclusión

A través de estos tres programas se comprobó de manera práctica cómo un proceso recibe sus argumentos por línea de comandos (`argc/argv`), cómo `execve()` y `execvp()` reemplazan por completo la imagen de un proceso sin crear uno nuevo, y cómo `fork()` permite que padre e hijo corran en paralelo sin bloquearse mutuamente cuando no se usa `wait()`. En los tres casos se aplicó el mismo patrón defensivo visto en clase: nunca dejar una llamada a la familia `exec` "suelta", sino envolverla en un bloque `if` que reporte el error con `perror()` y termine el proceso con `exit(EXIT_FAILURE)` — ya que, si la llamada tiene éxito, esa comprobación nunca llega a ejecutarse.

Referencias

- The Open Group. (2018). *execve — execute a file*. The Open Group Base Specifications Issue 7.
<https://pubs.opengroup.org/onlinepubs/9699919799/functions/execve.html>
- The Open Group. (2018). *fork — create a new process*. The Open Group Base Specifications Issue 7.
<https://pubs.opengroup.org/onlinepubs/9699919799/functions/fork.html>
- Stevens, W. R., & Rago, S. A. (2013). *Advanced programming in the UNIX environment* (3.^a ed.). Addison-Wesley.